

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Математическая логика и теория формальных языков

Отчёт о выполнении лабораторной работы №3

«Построение контекстно-свободной грамматики

подмножества естественного языка»

Вариант 35

Студент,

группа 5130201/30102

_____ Путята М. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Описание грамматики Futuro de indicativo	4
1.1 Примеры использования	4
1.2 Формы предложений	4
1.2.1 Утвердительные предложения	4
1.2.2 Отрицательные предложения	5
1.2.3 Вопросительные предложения	5
2 Математическое описание	6
2.1 Формальные грамматики	6
2.2 Формальное определение грамматики Futuro de indicativo	6
2.2.1 Правила вывода	7
2.2.2 Представление рассматриваемой грамматики в БНФ	11
2.2.3 Примеры предложений описанной грамматики	13
3 Особенности реализации	15
3.1 Структура TreeNode	15
3.2 Класс LanguageModel	15
3.2.1 Метод buildGrammar	16
3.2.2 Метод generateNode	20
3.2.3 Метод parseNode	21
3.2.4 Метод parseErrorTree	22
3.2.5 Обработчики текста	24
3.2.6 Метод checkAndPrintSentence	25
3.2.7 Метод tokensToString	27
3.3 Функция обработки пользовательского ввода	27
4 Результаты работы программы	30
Заключение	33
Список использованной литературы	34

Введение

В ходе выполнения данной лабораторной работы необходимо разработать контекстно-свободную грамматику для выбранного подмножества естественного языка. Для выполнения работы будет реализовано подмножество испанского языка, охватывающее простые предложения во времени Futuro de indicativo (будущее простое) в утвердительной, отрицательной и вопросительной формах.

Требуется, чтобы разрабатываемая программа могла как генерировать корректные предложения, соответствующие реализуемой КСГ, так и определять корректность введённых пользователем предложений.

Для выполнения поставленной задачи необходимо:

1. Изучить правила построения предложений, склонения существительных и спряжения глаголов в будущем простом времени испанского языка.
2. Выделить подмножество испанского языка будущего простого времени.
3. Формально описать грамматику данного языка и времени.
4. Составить БНФ для реализуемой контекстно-свободной грамматики.
5. Написать программную реализацию для созданной контекстно-свободной грамматики.
6. Создать инструменты для проверки корректности предложений и генерации корректных предложений.

1 Описание грамматики Futuro de indicativo

Испанское будущее простое время используется для описания действий, которые произойдут после момента речи. Оно характеризуется специфическими окончаниями, добавляемыми к инфинитиву глагола. Данные окончания обязательно должны соответствовать лицу и числу существительного. В таблице 1 представлены окончания для всех лиц и чисел подлежащего.

Таблица 1

Лицо	Единственное число	Множественное число
1-е (Я / Мы)	-é	-emos
2-е (Ты / Вы)	-ás	-éis
3-е (Он, Она / Они)	-á	-án

Также следует отметить, что существительные и прилагательные также должны быть согласованы между собой, но в отличие от существительного и глагола, они согласуются по роду и числу, то есть в испанском языке различаются прилагательные множественного числа для разных родов. Например в связке с существительным мужского рода множественного числа прилагательное "быстрые" будет переводиться как "rápidos", а для женского рода во множественном числе – "rápidas", но также есть и прилагательные-исключения, которые для обоих родов имеют одинаковое написание – "grandes", "interesantes", "importantes" и другие.

1.1 Примеры использования

Данная грамматика позволяет строить предложения, описывающие намерения или будущие события:

- **Yo hablaré con el profesor mañana.** – Я поговорю с учителем завтра.
- **Ellos comerán en el restaurante.** – Они будут есть в ресторане.

1.2 Формы предложений

Из написанных ниже форм предложений видно, что подлежащее является необязательной частью предложения, так как род и лицо глагола полностью совпадают с родом и лицом существительного, поэтому зачастую предложения строятся без подлежащего.

1.2.1 Утвердительные предложения

Строятся по схеме: [Подлежащее] + (Инфинитив + -é / -ás / -á / -emos / -éis / -án) / Неправильный глагол + [Дополнение / Определение / Обстоятельство] + .

Пример: Yo hablaré con el profesor mañana.

1.2.2 Отрицательные предложения

Имеют ту же схему, что и утвердительные предложения, но имеют частицу "no" перед глаголом.

Строятся по схеме: [Подлежащее] + no + (Инфинитив + -é / -ás / -á / -emos / -éis / -án) / Неправильный глагол + [Дополнение / Определение / Обстоятельство] + .

Пример: Tú no comerás el libro.

1.2.3 Вопросительные предложения

Используют обрамляющие знаки ¿ и ?, а также необязательные вопросительные слова в начале предложения.

Строятся по схеме: ¿ + [Вопросительное слово] + (Инфинитив + -é / -ás / -á / -emos / -éis / -án) / Неправильный глагол + [Подлежащее] + [Дополнение / Определение / Обстоятельство] + ?

Пример: ¿Cuándo comerá el gato?

"su", "nuestro", "vuestro", "todo", "poco", "mucho", "la", "una", "esta", "nuestra", "vuestra", "toda", "poca", "muchacha", "los", "unos", "estos", "mis", "tus", "sus", "nuestros", "vuestros", "todos", "pocos", "muchos", "las", "unas", "estas", "nuestras", "vuestras", "todas", "pocas", "muchas", "libro", "coche", "tiempo", "año", "trabajo", "amigo", "profesor", "perro", "gato", "árbol", "problema", "casa", "ciudad", "vida", "escuela", "amiga", "profesora", "mesa", "puerta", "flor", "noche", "mano", "libros", "coches", "tiempos", "años", "trabajos", "amigos", "profesores", "perros", "gatos", "árboles", "problemas", "casas", "ciudades", "vidas", "escuelas", "amigas", "profesoras", "mesas", "puertas", "flores", "noches", "manos", "bueno", "nuevo", "viejo", "pequeño", "hermoso", "grande", "interesante", "importante", "rojo", "rápido", "inteligente", "buena", "nueva", "vieja", "pequeña", "hermosa", "roja", "rápida", "buenos", "nuevos", "viejos", "pequeños", "hermosos", "grandes", "interesantes", "importantes", "rojos", "rápidos", "inteligentes", "buenas", "nuevas", "viejas", "pequeñas", "hermosas", "rojas", "rápidas", "que", "en", "a", "de", "con", "por", "para", "sin", "sobre", "desde", "hasta", "entre", "bajo", "é", "ás", "á", "emos", "éis", "án", "hablar", "comer", "vivir", "estudiar", "viajar", "escribir", "leer", "koppep", "correr", "abrir", "recibir", "cantar", "bailar", "trabajar", "tendr", "vendr", "saldr", "podr", "querr", "habr", "dir", "har", "sabr", "pondr", "valdr", "cabr", "lo", "la", "los", "las", "le", "les", "mañana", "pronto", "siempre", "nunca", "rápidamente", "bien", "mal", "aquí", "allí", "también", "probablemente", "seguramente", "quizás", "qué", "quién", "quiénes", "cuándo", "dónde", "adónde", "por qué", "cómo", "cuál", "cuáles", "cuánto", "cuánta", "cuántos", "cuántas" } – множество терминальных символов.

- $N = \{ \text{sentence, affirmative, negative, interrogative, subj_1sg, subj_2sg, subj_3sg, subj_1pl, subj_2pl, subj_3pl, noun_phrase_ms, noun_phrase_fs, noun_phrase_mp, noun_phrase_fp, det_ms, det_fs, det_mp, det_fp, noun_ms, noun_fs, noun_mp, noun_fp, adj_ms, adj_fs, adj_mp, adj_fp, postmod_ms, postmod_fs, postmod_mp, postmod_fp, prep_phrase_ms, prep_phrase_fs, prep_phrase_mp, prep_phrase_fp, relative_clause, verb_any, preposition, verb_1sg, verb_2sg, verb_3sg, verb_1pl, verb_2pl, verb_3pl, inf, regular_inf, irregular_base, complements, complement, direct_object, indirect_object, adverbial, question_word} \}$ – множество нетерминальных символов.
- $S = \text{sentence}$ – начальный символ грамматики.
- R – множество правил вывода вида $A \rightarrow \beta$, где $A \in N$ и $\beta \in \{T \cup N\}^+$. Правила вывода представлены следующем разделе.

2.2.1 Правила вывода

Типы предложений

```
1 <sentence> → <affirmative> | <negative> | <interrogative>
```

Утвердительные предложения

```
1 <affirmative> → <subj_1sg_opt> <verb_1sg> <complements> "."
2 <affirmative> → <subj_2sg_opt> <verb_2sg> <complements> "."
3 <affirmative> → <subj_3sg_opt> <verb_3sg> <complements> "."
4 <affirmative> → <subj_1pl_opt> <verb_1pl> <complements> "."
```

```

5 <affirmative> → <subj_2pl_opt> <verb_2pl> <complements> "."
6 <affirmative> → <subj_3pl_opt> <verb_3pl> <complements> "."
7
8 <subj_1sg_opt> → <subj_1sg> | ε
9 <subj_2sg_opt> → <subj_2sg> | ε
10 <subj_3sg_opt> → <subj_3sg> | ε
11 <subj_1pl_opt> → <subj_1pl> | ε
12 <subj_2pl_opt> → <subj_2pl> | ε
13 <subj_3pl_opt> → <subj_3pl> | ε

```

Отрицательные предложения

```

1 <negative> → <subj_1sg_opt> "no" <verb_1sg> <complements> "."
2 <negative> → <subj_2sg_opt> "no" <verb_2sg> <complements> "."
3 <negative> → <subj_3sg_opt> "no" <verb_3sg> <complements> "."
4 <negative> → <subj_1pl_opt> "no" <verb_1pl> <complements> "."
5 <negative> → <subj_2pl_opt> "no" <verb_2pl> <complements> "."
6 <negative> → <subj_3pl_opt> "no" <verb_3pl> <complements> "."

```

Вопросительные предложения

```

1 <interrogative> → "¿" <qword_opt> <verb_1sg> <subj_1sg_opt> <
  complements> "?"
2 <interrogative> → "¿" <qword_opt> <verb_2sg> <subj_2sg_opt> <
  complements> "?"
3 <interrogative> → "¿" <qword_opt> <verb_3sg> <subj_3sg_opt> <
  complements> "?"
4 <interrogative> → "¿" <qword_opt> <verb_1pl> <subj_1pl_opt> <
  complements> "?"
5 <interrogative> → "¿" <qword_opt> <verb_2pl> <subj_2pl_opt> <
  complements> "?"
6 <interrogative> → "¿" <qword_opt> <verb_3pl> <subj_3pl_opt> <
  complements> "?"
7
8 <qword_opt> → <question_word> | ε

```

Подлежащие

```

1 <subj_1sg> → "yo"
2 <subj_2sg> → "tú"
3 <subj_3sg> → "él" | "ella" | "usted" | <noun_phrase_ms> | <
  noun_phrase_fs>
4 <subj_1pl> → "nosotros" | "nosotras"
5 <subj_2pl> → "vosotros" | "vosotras"
6 <subj_3pl> → "ellos" | "ellas" | "ustedes" | <noun_phrase_mp> | <
  noun_phrase_fp>

```

Именные группы

```

1 <noun_phrase_ms> → <det_ms_opt> <adj_ms_seq> <noun_ms> <
  postmod_ms_seq>

```



```

2    <noun_phrase_fs> → <det_fs_opt> <adj_fs_seq> <noun_fs> <
    postmod_fs_seq>
3    <noun_phrase_mp> → <det_mp_opt> <adj_mp_seq> <noun_mp> <
    postmod_mp_seq>
4    <noun_phrase_fp> → <det_fp_opt> <adj_fp_seq> <noun_fp> <
    postmod_fp_seq>
5
6    <det_ms_opt> → <det_ms> | ε
7    <det_fs_opt> → <det_fs> | ε
8    <det_mp_opt> → <det_mp> | ε
9    <det_fp_opt> → <det_fp> | ε
10
11   <adj_ms_seq> → <adj_ms> <adj_ms_seq> | ε
12   <adj_fs_seq> → <adj_fs> <adj_fs_seq> | ε
13   <adj_mp_seq> → <adj_mp> <adj_mp_seq> | ε
14   <adj_fp_seq> → <adj_fp> <adj_fp_seq> | ε
15
16   <postmod_ms_seq> → <postmod_ms> <postmod_ms_seq> | ε
17   <postmod_fs_seq> → <postmod_fs> <postmod_fs_seq> | ε
18   <postmod_mp_seq> → <postmod_mp> <postmod_mp_seq> | ε
19   <postmod_fp_seq> → <postmod_fp> <postmod_fp_seq> | ε

```

Определители, существительные, прилагательные

```

1    <det_ms> → "el" | "un" | "este" | "mi" | "tu" | "su" | "nuestro" | "
    vuestro" | "todo" | "poco" | "mucho"
2    <det_fs> → "la" | "una" | "esta" | "mi" | "tu" | "su" | "nuestra" |
    "vuestra" | "toda" | "poca" | "mucho"
3    <det_mp> → "los" | "unos" | "estos" | "mis" | "tus" | "sus" | "
    nuestros" | "vuestros" | "todos" | "pocos" | "muchos"
4    <det_fp> → "las" | "unas" | "estas" | "mis" | "tus" | "sus" | "
    nuestras" | "vuestras" | "todas" | "pocas" | "muchas"
5
6    <noun_ms> → "libro" | "coche" | "tiempo" | "año" | "trabajo" | "
    amigo" | "profesor" | "perro" | "gato" | "árbol" | "problema"
7    <noun_fs> → "casa" | "ciudad" | "vida" | "escuela" | "amiga" | "
    profesora" | "mesa" | "puerta" | "flor" | "noche" | "mano"
8    <noun_mp> → "libros" | "coches" | "tiempos" | "años" | "trabajos" |
    "amigos" | "profesores" | "perros" | "gatos" | "árboles" | "
    problemas"
9    <noun_fp> → "casas" | "ciudades" | "vidas" | "escuelas" | "amigas" |
    "profesoras" | "mesas" | "puertas" | "flores" | "noches" | "
    manos"
10
11   <adj_ms> → "bueno" | "nuevo" | "viejo" | "pequeño" | "hermoso" | "
    grande" | "interesante" | "importante" | "rojo" | "rápido" | "
    inteligente"
12   <adj_fs> → "buena" | "nueva" | "vieja" | "pequeña" | "hermosa" | "
    grande" | "interesante" | "importante" | "roja" | "rápida" | "
    inteligente"
13   <adj_mp> → "buenos" | "nuevos" | "viejos" | "pequeños" | "hermosos"
    | "grandes" | "interesantes" | "importantes" | "rojos" | "rápidos
    " | "inteligentes"
14   <adj_fp> → "buenas" | "nuevas" | "viejas" | "pequeñas" | "hermosas"
    | "grandes" | "interesantes" | "importantes" | "rojas" | "rápidas

```

```
" | "inteligentes"
```

Постмодификаторы

```
1 <postmod_ms> → <prep_phrase_ms> | <relative_clause>
2 <postmod_fs> → <prep_phrase_fs> | <relative_clause>
3 <postmod_mp> → <prep_phrase_mp> | <relative_clause>
4 <postmod_fp> → <prep_phrase_fp> | <relative_clause>
5
6 <prep_phrase_ms> → <preposition> <noun_phrase_ms>
7 <prep_phrase_fs> → <preposition> <noun_phrase_fs>
8 <prep_phrase_mp> → <preposition> <noun_phrase_mp>
9 <prep_phrase_fp> → <preposition> <noun_phrase_fp>
10
11 <relative_clause> → "que" <verb_any> <complements>
12 <verb_any> → <verb_1sg> | <verb_2sg> | <verb_3sg> | <verb_1pl> | <
    verb_2pl> | <verb_3pl>
13 <preposition> → "en" | "a" | "de" | "con" | "por" | "para" | "sin" |
    "sobre" | "desde" | "hasta" | "entre" | "bajo"
```

Глаголы

```
1 <verb_1sg> → <inf> "é"
2 <verb_2sg> → <inf> "ás"
3 <verb_3sg> → <inf> "á"
4 <verb_1pl> → <inf> "emos"
5 <verb_2pl> → <inf> "éis"
6 <verb_3pl> → <inf> "án"
7
8 <inf> → <regular_inf> | <irregular_base>
9 <regular_inf> → "hablar" | "comer" | "vivir" | "estudiar" | "viajar"
    | "escribir" | "leer" | "correr" | "abrir" | "recibir" | "cantar"
    | "bailar" | "trabajar"
10 <irregular_base> → "tendr" | "vendr" | "saldr" | "podr" | "querr" |
    "habr" | "dir" | "har" | "sabr" | "pondr" | "valdr" | "cabr"
```

Дополнения и обстоятельства

```
1 <complements> → <complement> <complements> | <complements> <
    complement> | ε
2 <complement> → <direct_object> | <indirect_object> | <adverbial>
3
4 <direct_object> → <noun_phrase_ms> | <noun_phrase_fs> | <
    noun_phrase_mp> | <noun_phrase_fp> | "lo" | "la" | "los" | "las"
5 <indirect_object> → "a" <noun_phrase_ms> | "a" <noun_phrase_fs> | "a"
    <noun_phrase_mp> | "a" <noun_phrase_fp> | "le" | "les"
6
7 <adverbial> → "mañana" | "pronto" | "siempre" | "nunca" | "rá
    pidamente" | "bien" | "mal" | "aquí" | "allí" | "también" | "
    probablemente" | "seguramente" | "quizás"
```

Вопросительные слова

```
1 <question_word> → "qué" | "quién" | "quiénes" | "cuándo" | "dónde" |  
  "adónde" | "por_qué" | "cómo" | "cuál" | "cuáles" | "cuánto" | "  
  cuánta" | "cuántos" | "cuántas"
```

2.2.2 Представление рассматриваемой грамматики в БНФ

Форма Бэкуса-Наура (БНФ) – форма для представления контекстно-свободных грамматик. В данной форме продукции имеющие одинаковые левые части объединены с помощью объединения правых частей через знак «|».

Для БНФ также существуют расширения позволяющие добавлять регулярные конструкции такие как группировка (a | {b} | c), итерация {a} и опциональность [a].

На листинге 1 представлена грамматика подмножества испанского языка в будущем простом времени в расширенной БНФ.

Листинг 1. Грамматика подмножества языка в БНФ

```
1 <sentence> ::= <affirmative> | <negative> | <interrogative>  
2  
3 УТВЕРДИТЕЛЬНОЕ  
4 <affirmative> ::= [<subj_1sg>] <verb_1sg> <complements> "."  
5 | [<subj_2sg>] <verb_2sg> <complements> "."  
6 | [<subj_3sg>] <verb_3sg> <complements> "."  
7 | [<subj_1pl>] <verb_1pl> <complements> "."  
8 | [<subj_2pl>] <verb_2pl> <complements> "."  
9 | [<subj_3pl>] <verb_3pl> <complements> "."  
10  
11 ОТРИЦАТЕЛЬНОЕ  
12 <negative> ::= [<subj_1sg>] "no" <verb_1sg> <complements> "."  
13 | [<subj_2sg>] "no" <verb_2sg> <complements> "."  
14 | [<subj_3sg>] "no" <verb_3sg> <complements> "."  
15 | [<subj_1pl>] "no" <verb_1pl> <complements> "."  
16 | [<subj_2pl>] "no" <verb_2pl> <complements> "."  
17 | [<subj_3pl>] "no" <verb_3pl> <complements> "."  
18  
19 ВОПРОСИТЕЛЬНОЕ  
20 <interrogative> ::= "¿" [<question_word>] <verb_1sg> [<subj_1sg>] <  
  complements> "?"  
21 | "¿" [<question_word>] <verb_2sg> [<subj_2sg>] <complements> "?"  
22 | "¿" [<question_word>] <verb_3sg> [<subj_3sg>] <complements> "?"  
23 | "¿" [<question_word>] <verb_1pl> [<subj_1pl>] <complements> "?"  
24 | "¿" [<question_word>] <verb_2pl> [<subj_2pl>] <complements> "?"  
25 | "¿" [<question_word>] <verb_3pl> [<subj_3pl>] <complements> "?"  
26  
27 ПОДЛЕЖАЩИЕ  
28 <subj_1sg> ::= "yo"  
29 <subj_2sg> ::= "tú"  
30 <subj_3sg> ::= "él" | "ella" | "usted" | <noun_phrase_ms> | <  
  noun_phrase_fs>
```

```

31 <subj_1pl> ::= "nosotros" | "nosotras"
32 <subj_2pl> ::= "vosotros" | "vosotras"
33 <subj_3pl> ::= "ellos" | "ellas" | "ustedes" | <noun_phrase_mp> | <
    noun_phrase_fp>
34 ИМЕННЫЕ ГРУППЫ
35 <noun_phrase_ms> ::= [<det_ms>] {<adj_ms>} <noun_ms> {<postmod_ms>}
36 <noun_phrase_fs> ::= [<det_fs>] {<adj_fs>} <noun_fs> {<postmod_fs>}
37 <noun_phrase_mp> ::= [<det_mp>] {<adj_mp>} <noun_mp> {<postmod_mp>}
38 <noun_phrase_fp> ::= [<det_fp>] {<adj_fp>} <noun_fp> {<postmod_fp>}
39
40 ОПРЕДЕЛИТЕЛИ
41 <det_ms> ::= "el" | "un" | "este" | "mi" | "tu" | "su" | "nuestro" | "
    vuestro" | "todo" | "poco" | "mucho"
42 <det_fs> ::= "la" | "una" | "esta" | "mi" | "tu" | "su" | "nuestra" | "
    vuestra" | "toda" | "poca" | "muchas"
43 <det_mp> ::= "los" | "unos" | "estos" | "mis" | "tus" | "sus" | "
    nuestros" | "vuestros" | "todos" | "pocos" | "muchos"
44 <det_fp> ::= "las" | "unas" | "estas" | "mis" | "tus" | "sus" | "
    nuestras" | "vuestras" | "todas" | "pocas" | "muchas"
45
46 СУЩЕСТВИТЕЛЬНЫЕ
47 <noun_ms> ::= "libro" | "coche" | "tiempo" | "año" | "trabajo" | "amigo"
    | "profesor" | "perro" | "gato" | "árbol" | "problema"
48 <noun_fs> ::= "casa" | "ciudad" | "vida" | "escuela" | "amiga" | "
    profesora" | "mesa" | "puerta" | "flor" | "noche" | "mano"
49 <noun_mp> ::= "libros" | "coches" | "tiempos" | "años" | "trabajos" | "
    amigos" | "profesores" | "perros" | "gatos" | "árboles" | "problemas"
50 <noun_fp> ::= "casas" | "ciudades" | "vidas" | "escuelas" | "amigas" | "
    profesoras" | "mesas" | "puertas" | "flores" | "noches" | "manos"
51
52 ПРИЛАГАТЕЛЬНЫЕ
53 <adj_ms> ::= "bueno" | "nuevo" | "viejo" | "pequeño" | "hermoso" | "
    grande" | "interesante" | "importante" | "rojo" | "rápido" | "
    inteligente"
54 <adj_fs> ::= "buena" | "nueva" | "vieja" | "pequeña" | "hermosa" | "
    grande" | "interesante" | "importante" | "roja" | "rápida" | "
    inteligente"
55 <adj_mp> ::= "buenos" | "nuevos" | "viejos" | "pequeños" | "hermosos" |
    "grandes" | "interesantes" | "importantes" | "rojos" | "rápidos" | "
    inteligentes"
56 <adj_fp> ::= "buenas" | "nuevas" | "viejas" | "pequeñas" | "hermosas" |
    "grandes" | "interesantes" | "importantes" | "rojas" | "rápidas" | "
    inteligentes"
57
58 ПОСТМОДИФИКАТОРЫ
59 <postmod_ms> ::= <prep_phrase_ms> | <relative_clause>
60 <postmod_fs> ::= <prep_phrase_fs> | <relative_clause>
61 <postmod_mp> ::= <prep_phrase_mp> | <relative_clause>
62 <postmod_fp> ::= <prep_phrase_fp> | <relative_clause>
63 <prep_phrase_ms> ::= <preposition> <noun_phrase_ms>
64 <prep_phrase_fs> ::= <preposition> <noun_phrase_fs>
65 <prep_phrase_mp> ::= <preposition> <noun_phrase_mp>
66 <prep_phrase_fp> ::= <preposition> <noun_phrase_fp>
67 <relative_clause> ::= "que" <verb_any> <complements>
68 <verb_any> ::= <verb_1sg> | <verb_2sg> | <verb_3sg> | <verb_1pl> | <
    verb_2pl> | <verb_3pl>

```

```

69 <preposition> ::= "en" | "a" | "de" | "con" | "por" | "para" | "sin" | "
    sobre" | "desde" | "hasta" | "entre" | "bajo"
70
71 ГЛАГОЛ В БУДУЩЕМ ПРОСТОМ
72 <verb_1sg> ::= <inf> "é"
73 <verb_2sg> ::= <inf> "ás"
74 <verb_3sg> ::= <inf> "á"
75 <verb_1pl> ::= <inf> "emos"
76 <verb_2pl> ::= <inf> "éis"
77 <verb_3pl> ::= <inf> "án"
78 <inf> ::= <regular_inf> | <irregular_base>
79 <regular_inf> ::= "hablar" | "comer" | "vivir" | "estudiar" | "viajar" |
    "escribir" | "leer" | "correr" | "abrir" | "recibir" | "cantar" | "
    bailar" | "trabajar"
80 <irregular_base> ::= "tendr" | "vendr" | "saldr" | "podr" | "querr" | "
    habr" | "dir" | "har" | "sabr" | "pondr" | "valdr" | "cabr"
81
82 ДОПОЛНЕНИЯ И ОБСТОЯТЕЛЬСТВА
83 <complements> ::= <complement> <complements> | <complements> <complement
    > | $\epsilon$
84 <complement> ::= <direct_object> | <indirect_object> | <adverbial>
85 <direct_object> ::= <noun_phrase_ms> | <noun_phrase_fs> | <
    noun_phrase_mp> | <noun_phrase_fp> | "lo" | "la" | "los" | "las"
86 <indirect_object> ::= "a" <noun_phrase_ms> | "a" <noun_phrase_fs> | "a"
    <noun_phrase_mp> | "a" <noun_phrase_fp> | "le" | "les"
87 <adverbial> ::= "mañana" | "pronto" | "siempre" | "nunca" | "rápidamente
    " | "bien" | "mal" | "aquí" | "allí" | "también" | "probablemente" |
    "seguramente" | "quizás"
88
89 ВОПРОСИТЕЛЬНЫЕ СЛОВА
90 <question_word> ::= "qué" | "quién" | "quiénes" | "cuándo" | "dónde" | "
    adónde" | "por qué" | "cómo" | "cuál" | "cuáles" | "cuánto" | "cuánta
    " | "cuántos" | "cuántas"

```

2.2.3 Примеры предложений описанной грамматики

- Утвердительные

- Hablaré aquí ciudades que leerán le.
- Nosotras no bailaremos allí roja amiga mañana.

- Отрицательные

- Nosotras no bailaremos allí mañana.
- No leerás nuevas profesoras por sus rápidas ciudades.

- Вопросительные

- ¿Cuántas bailaremos nosotras pronto?
- ¿Adónde sabréis vosotras a mi pequeño viejo libro?

Данная грамматика не может быть упрощена до автоматной, так как:

- в ней есть косвенная рекурсия:

$$\langle \text{noun_phrase_ms} \rangle ::= [\langle \text{det_ms} \rangle] \{ \langle \text{adj_ms} \rangle \} \langle \text{noun_ms} \rangle \{ \langle \text{postmod_ms} \rangle \}$$

$$\langle \text{postmod_ms} \rangle ::= \langle \text{prep_phrase_ms} \rangle \mid \langle \text{relative_clause} \rangle$$

$$\langle \text{prep_phrase_ms} \rangle ::= \langle \text{preposition} \rangle \langle \text{noun_phrase_ms} \rangle$$

- в ней есть двусторонняя рекурсия:

$$\langle \text{complements} \rangle ::= \langle \text{complement} \rangle \langle \text{complements} \rangle \mid \langle \text{complements} \rangle \langle \text{complement} \rangle \mid \epsilon$$

На рисунке 1 представлен пример дерева построения предложения по реализованной грамматике для предложения "¿Cabremos a años los gatos probablemente mañana?".

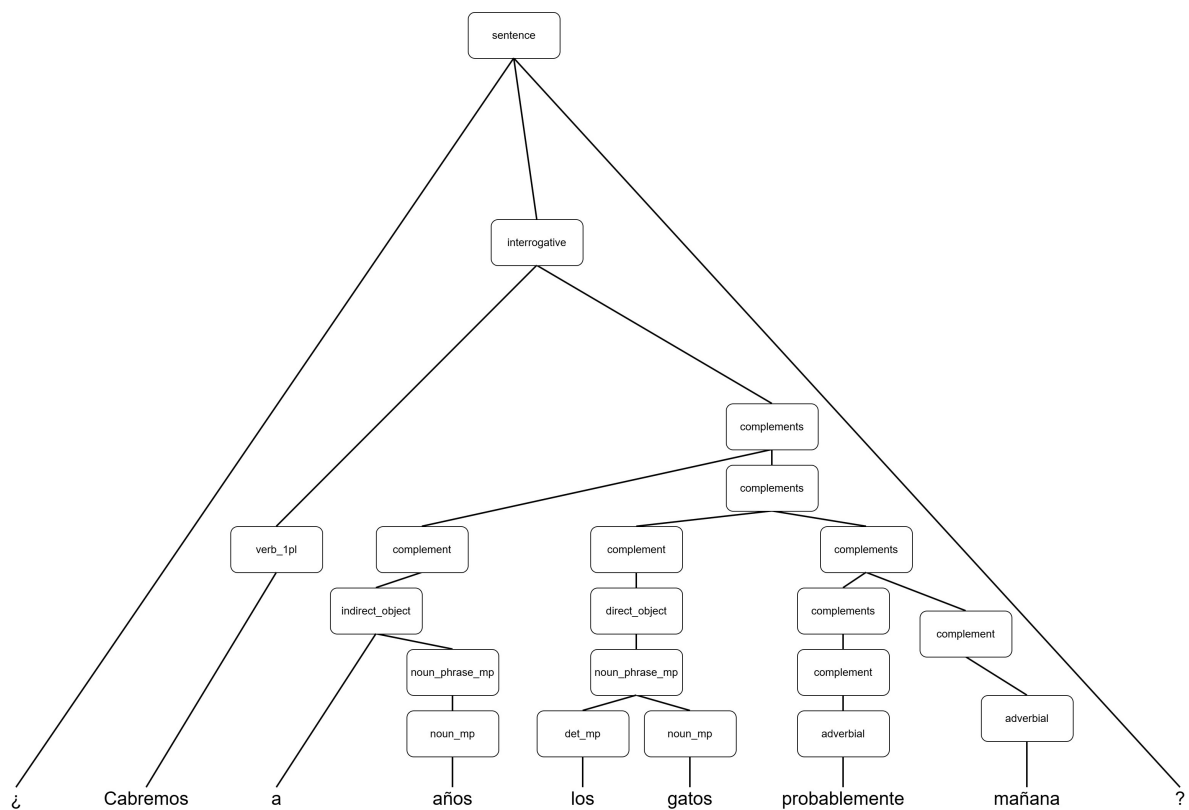


Рис. 1. Дерево построения предложения

3 Особенности реализации

3.1 Структура TreeNode

Данная структура позволяет хранить узел N-арного дерева. Объекты этой структуры динамически создаются во время рекурсивного спуска в методе `parseNode` и `parseErrorTree` для формирования синтаксического дерева. В случае если ветка разбора оказывается тупиковой, узлы уничтожаются, иначе – полученное дерево (экземпляр, представляющий корень) передаётся в функцию отрисовки дерева в консоли. На листинге 2 представлен код определения данной структуры.

Листинг 2. Структура `TreeNode`

```
1 struct TreeNode {
2     string symbol;
3     vector<TreeNode*> children;
4
5     TreeNode(string s) : symbol(s) {}
6
7     ~TreeNode() {
8         for (auto c : children) {
9             delete c;
10        }
11    }
12};
```

Назначение полей

- `symbol` – строковое представление узла для вывода в консоль. Например, "complements" или "noun_phrase_np";
- `children` – вектор указателей на дочерние узлы дерева.

3.2 Класс `LanguageModel`

Данный класс реализует логику КС грамматики. Использование класса `LanguageModel` позволяет создавать грамматики для любых КС языков с помощью простого добавления правил вывода с помощью метода `addRule`. Класс объединяет в себе базу правил вывода, генератор предложений и синтаксический анализатор. Код определения данного класса представлен на листинге 3.

Листинг 3. Класс `LanguageModel`

```
1 class LanguageModel {
2 private:
3     Grammar rules;
4     vector<string> vocabulary;
5     size_t max_parse_pos = 0;
6
7     void addRule(const string& l, const vector<vector<string>>& r) {
8         rules[l] = r;
```

```

9      }
10
11     void buildGrammar();
12     void generateNode(const string& symbol, vector<string>& output, int
        depth = 0);
13     bool parseNode(const vector<string>& tokens, size_t& pos, const
        string& symbol, TreeNode*& out_node);
14     int parseErrorTree(const vector<string>& tokens, size_t& pos, const
        string& symbol, TreeNode*& out_node);
15     void replaceAll(string& str, const string& from, const string& to);
16     void printTree(TreeNode* node, string prefix = "", bool isLast = true
        );
17
18     bool isCapitalized(const string& str);
19     string lowercaseUtf8(const string& str);
20     string capitalizeUtf8(const string& str);
21
22 public:
23     LanguageModel();
24
25     string tokensToString(const vector<string>& tokens);
26     vector<string> generateTokenList();
27     void checkAndPrintSentence(string text, int tree_flag = 0);
28 };

```

Назначение полей

- rules – словарь, алфавит грамматики.
- vocabulary – множество всех возможных терминалов заданного языка.
- max_parse_pos – переменная, хранящая самый длинный индекс слова при разборе и используемая для возврата назад по предложению.

Конструктор класса Реализованный конструктор для данного класса используется для задания седа генерации случайных чисел и заполнения правил вывода экземпляра. Код конструктора представлен на листинге 4.

Вход: набор правил вывода и текущее время.

Выход: экземпляр класса LanguageModel.

Листинг 4. Конструктор класса LanguageModel

```

1 LanguageModel() {
2     srand((unsigned)time(0));
3     buildGrammar();
4 }

```

3.2.1 Метод buildGrammar

Данный метод используется в конструкторе класса LanguageModel для задания правил вывода и формирования алфавита. buildGrammar содержит множественные

вызовы метода `addRule` для определения каждого правила. Также для уменьшения размера исходного кода при определении правил вывода, содержащих терминалы глаголов, используется цикл, который формирует все возможные глаголы будущего времени с помощью комбинации каждого глагола с каждым возможным окончанием. На листинге 5 можно увидеть код данного метода.

Вход: набор правил вывода грамматики испанского языка.

Выход: заполненные правила вывода и алфавит класса `LanguageModel`.

Листинг 5. Код метода `buildGrammar`

```

1 void buildGrammar() {
2     addRule("subj_1sg", { {"yo"} });
3     addRule("subj_2sg", { {"tú"} });
4     addRule("subj_3sg", { {"él"}, {"ella"}, {"usted"}, {"noun_phrase_ms"}
5         , {"noun_phrase_fs"} });
6     addRule("subj_1pl", { {"nosotros"}, {"nosotras"} });
7     addRule("subj_2pl", { {"vosotros"}, {"vosotras"} });
8     addRule("subj_3pl", { {"ellos"}, {"ellas"}, {"ustedes"}, {""
9         noun_phrase_mp"}, {"noun_phrase_fp"} });
10
11     addRule("opt_subj_1sg", { {"subj_1sg"}, {} });
12     addRule("opt_subj_2sg", { {"subj_2sg"}, {} });
13     addRule("opt_subj_3sg", { {"subj_3sg"}, {} });
14     addRule("opt_subj_1pl", { {"subj_1pl"}, {} });
15     addRule("opt_subj_2pl", { {"subj_2pl"}, {} });
16     addRule("opt_subj_3pl", { {"subj_3pl"}, {} });
17
18     addRule("sentence", { {"affirmative"}, {"negative"}, {"interrogative"}
19         });
20
21     addRule("affirmative", {
22         {"opt_subj_1sg", "verb_1sg", "complements", "."},
23         {"opt_subj_2sg", "verb_2sg", "complements", "."},
24         {"opt_subj_3sg", "verb_3sg", "complements", "."},
25         {"opt_subj_1pl", "verb_1pl", "complements", "."},
26         {"opt_subj_2pl", "verb_2pl", "complements", "."},
27         {"opt_subj_3pl", "verb_3pl", "complements", "."}
28     });
29
30     addRule("negative", {
31         {"opt_subj_1sg", "no", "verb_1sg", "complements", "."},
32         {"opt_subj_2sg", "no", "verb_2sg", "complements", "."},
33         {"opt_subj_3sg", "no", "verb_3sg", "complements", "."},
34         {"opt_subj_1pl", "no", "verb_1pl", "complements", "."},
35         {"opt_subj_2pl", "no", "verb_2pl", "complements", "."},
36         {"opt_subj_3pl", "no", "verb_3pl", "complements", "."}
37     });
38
39     addRule("opt_qword", { {"question_word"}, {} });
40     addRule("interrogative", {
41         {"¿", "opt_qword", "verb_1sg", "opt_subj_1sg", "complements", "?"},
42         {"¿", "opt_qword", "verb_2sg", "opt_subj_2sg", "complements", "?"},
43         {"¿", "opt_qword", "verb_3sg", "opt_subj_3sg", "complements", "?"}
44     });

```

```

41     {"¿", "opt_qword", "verb_1pl", "opt_subj_1pl", "complements", "?"}
42     {"¿", "opt_qword", "verb_2pl", "opt_subj_2pl", "complements", "?"}
43     {"¿", "opt_qword", "verb_3pl", "opt_subj_3pl", "complements", "?"}
44     });
45
46     addRule("noun_phrase_ms", { {"opt_det_ms", "opt_list_adj_ms", "
47         noun_ms", "list_postmod_ms"} });
48     addRule("noun_phrase_fs", { {"opt_det_fs", "opt_list_adj_fs", "
49         noun_fs", "list_postmod_fs"} });
50     addRule("noun_phrase_mp", { {"opt_det_mp", "opt_list_adj_mp", "
51         noun_mp", "list_postmod_mp"} });
52     addRule("noun_phrase_fp", { {"opt_det_fp", "opt_list_adj_fp", "
53         noun_fp", "list_postmod_fp"} });
54
55     addRule("opt_det_ms", { {"det_ms"}, {} }); addRule("opt_det_fs", { {"
56         det_fs"}, {} });
57     addRule("opt_det_mp", { {"det_mp"}, {} }); addRule("opt_det_fp", { {"
58         det_fp"}, {} });
59
60     addRule("opt_list_adj_ms", { {"list_adj_ms"}, {} });
61     addRule("opt_list_adj_fs", { {"list_adj_fs"}, {} });
62     addRule("opt_list_adj_mp", { {"list_adj_mp"}, {} });
63     addRule("opt_list_adj_fp", { {"list_adj_fp"}, {} });
64
65     addRule("list_adj_ms", { {"adj_ms", "list_adj_ms", {"adj_ms"} });
66     addRule("list_adj_fs", { {"adj_fs", "list_adj_fs", {"adj_fs"} });
67     addRule("list_adj_mp", { {"adj_mp", "list_adj_mp", {"adj_mp"} });
68     addRule("list_adj_fp", { {"adj_fp", "list_adj_fp", {"adj_fp"} });
69
70     addRule("list_postmod_ms", { {"postmod_ms", "list_postmod_ms", {} })
71     ;
72     addRule("list_postmod_fs", { {"postmod_fs", "list_postmod_fs", {} })
73     ;
74     addRule("list_postmod_mp", { {"postmod_mp", "list_postmod_mp", {} })
75     ;
76     addRule("list_postmod_fp", { {"postmod_fp", "list_postmod_fp", {} })
77     ;
78
79     addRule("det_ms", { {"el"}, {"un"}, {"este"}, {"mi"}, {"tu"}, {"su"},
80         {"nuestro"}, {"vuestro"}, {"todo"}, {"poco"}, {"mucho"} });
81     addRule("det_fs", { {"la"}, {"una"}, {"esta"}, {"mi"}, {"tu"}, {"su"},
82         {"nuestra"}, {"vuestra"}, {"toda"}, {"poca"}, {"mucha"} });
83     addRule("det_mp", { {"los"}, {"unos"}, {"estos"}, {"mis"}, {"tus"}, {"
84         sus"}, {"nuestros"}, {"vuestros"}, {"todos"}, {"pocos"}, {"muchos"}
85     });
86     addRule("det_fp", { {"las"}, {"unas"}, {"estas"}, {"mis"}, {"tus"}, {"
87         sus"}, {"nuestras"}, {"vuestras"}, {"todas"}, {"pocas"}, {"muchas"}
88     });
89
90     addRule("noun_ms", { {"libro"}, {"coche"}, {"tiempo"}, {"año"}, {"
91         trabajo"}, {"amigo"}, {"profesor"}, {"perpo"}, {"gato"}, {"árbol"},
92         {"problema"} });
93     addRule("noun_fs", { {"casa"}, {"ciudad"}, {"vida"}, {"escuela"}, {"

```

```

    amiga"} , {"profesora"} , {"mesa"} , {"puerta"} , {"flor"} , {"noche"} ,
    {"mano"} }));
76 addRule("noun_mp", { {"libros"} , {"coches"} , {"tiempos"} , {"años"} , {
    "trabajos"} , {"amigos"} , {"profesores"} , {"perros"} , {"gatos"} , {"
    árboles"} , {"problemas"} }));
77 addRule("noun_fp", { {"casas"} , {"ciudades"} , {"vidas"} , {"escuelas"}
    , {"amigas"} , {"profesoras"} , {"mesas"} , {"puertas"} , {"flores"} ,
    {"noches"} , {"manos"} }));
78
79 addRule("adj_ms", { {"bueno"} , {"nuevo"} , {"viejo"} , {"pequeño"} , {"
    hermoso"} , {"grande"} , {"interesante"} , {"importante"} , {"rojo"} ,
    {"rápido"} , {"inteligente"} }));
80 addRule("adj_fs", { {"buena"} , {"nueva"} , {"vieja"} , {"pequeña"} , {"
    hermosa"} , {"grande"} , {"interesante"} , {"importante"} , {"roja"} ,
    {"rápida"} , {"inteligente"} }));
81 addRule("adj_mp", { {"buenos"} , {"nuevos"} , {"viejos"} , {"pequeños"} ,
    {"hermosos"} , {"grandes"} , {"interesantes"} , {"importantes"} , {"
    rojos"} , {"rápidos"} , {"inteligentes"} }));
82 addRule("adj_fp", { {"buenas"} , {"nuevas"} , {"viejas"} , {"pequeñas"} ,
    {"hermosas"} , {"grandes"} , {"interesantes"} , {"importantes"} , {"
    rojas"} , {"rápidas"} , {"inteligentes"} }));
83
84 addRule("postmod_ms", { {"prep_phrase_ms"} , {"relative_clause"} }));
85 addRule("postmod_fs", { {"prep_phrase_fs"} , {"relative_clause"} }));
86 addRule("postmod_mp", { {"prep_phrase_mp"} , {"relative_clause"} }));
87 addRule("postmod_fp", { {"prep_phrase_fp"} , {"relative_clause"} }));
88
89 addRule("prep_phrase_ms", { {"preposition"} , {"noun_phrase_ms"} }));
90 addRule("prep_phrase_fs", { {"preposition"} , {"noun_phrase_fs"} }));
91 addRule("prep_phrase_mp", { {"preposition"} , {"noun_phrase_mp"} }));
92 addRule("prep_phrase_fp", { {"preposition"} , {"noun_phrase_fp"} }));
93
94 addRule("relative_clause", { {"que"} , {"verb_any"} , {"complements"} }));
95 addRule("verb_any", { {"verb_1sg"} , {"verb_2sg"} , {"verb_3sg"} , {"
    verb_1pl"} , {"verb_2pl"} , {"verb_3pl"} }));
96
97 addRule("preposition", { {"en"} , {"a"} , {"de"} , {"con"} , {"por"} , {"
    para"} , {"sin"} , {"sobre"} , {"desde"} , {"hasta"} , {"entre"} , {"
    bajo"} }));
98
99 vector<string> inf = { "hablar" , "comer" , "vivir" , "estudiar" , "
    viajar" , "escribir" , "leer" , "correr" , "abrir" , "recibir" , "cantar
    " , "bailar" , "trabajar" };
100 vector<string> irr = { "tendr" , "vendr" , "saldr" , "podr" , "querr" , "
    habr" , "dir" , "har" , "sabr" , "pondr" , "valdr" , "cabr" };
101 vector<string> bases = inf;
102 bases.insert(bases.end(), irr.begin(), irr.end());
103
104 vector<vector<string>> v_1sg, v_2sg, v_3sg, v_1pl, v_2pl, v_3pl;
105 for (const string& b : bases) {
106     v_1sg.push_back({ b + "é" }); v_2sg.push_back({ b + "ás" });
107     v_3sg.push_back({ b + "á" }); v_1pl.push_back({ b + "emos" });
108     v_2pl.push_back({ b + "éis" }); v_3pl.push_back({ b + "án" });
109 }
110 addRule("verb_1sg", v_1sg); addRule("verb_2sg", v_2sg);
111 addRule("verb_3sg", v_3sg); addRule("verb_1pl", v_1pl);

```

```

112 addRule("verb_2pl", v_2pl); addRule("verb_3pl", v_3pl);
113
114 addRule("complements", { {"complement", "complements"}, {"complements",
115     "complement"}, {} });
116
117 addRule("direct_object", { {"noun_phrase_ms"}, {"noun_phrase_fs"}, {"noun_phrase_mp"}, {"noun_phrase_fp"}, {"lo"}, {"la"}, {"los"}, {"las"} });
118
119 addRule("indirect_object", { {"a", "noun_phrase_ms"}, {"a", "noun_phrase_fs"}, {"a", "noun_phrase_mp"}, {"a", "noun_phrase_fp"}, {"le"}, {"les"} });
120
121 addRule("adverbial", { {"mañana"}, {"pronto"}, {"siempre"}, {"nunca"}, {"rápidamente"}, {"bien"}, {"mal"}, {"aquí"}, {"allí"}, {"también"}, {"probablemente"}, {"seguramente"}, {"quizás"} });
122
123 addRule("question_word", { {"qué"}, {"quién"}, {"quiénes"}, {"cuándo"}, {"dónde"}, {"adónde"}, {"por_qué"}, {"cómo"}, {"cuál"}, {"cuáles"}, {"cuánto"}, {"cuánta"}, {"cuántos"}, {"cuántas"} });
124
125
126
127
128
129 set<string> terms;
130 for (const auto& pair : rules) {
131     for (const auto& prod : pair.second) {
132         for (const auto& sym : prod) {
133             if (rules.find(sym) == rules.end() && sym != "." && sym != "?" && sym != ";") {
134                 terms.insert(sym);
135             }
136         }
137     }
138 }
139 vocabulary.assign(terms.begin(), terms.end());
140 }

```

3.2.2 Метод generateNode

Данный метода реализует алгоритм рекурсивного вывода и используется для генерации синтаксически корректных предложений на заданном языке. Метод generateNode начинает обход с нетерминала sentence и случайным образом выбирает одно из доступных правил вывода пока не дойдёт до состояния, при котором останутся только терминалы. Код данного метода расположен на листинге 6.

Вход: текущий нетерминал, который нужно раскрыть и текущая глубина рекурсии.

Выход: обновлённый вектор символов предложения.

Листинг 6. Код метода generateNode

```

1 void generateNode(const string& symbol, vector<string>& output, int
  depth = 0) {
2   if (rules.find(symbol) == rules.end()) {
3     output.push_back(symbol);
4     return;
5   }
6   const auto& productions = rules[symbol];
7   if (productions.empty()) return;
8
9   int idx = 0;
10  if (productions.size() > 1 && depth < 16) {
11    idx = (rand() % 100 < (1 - depth / 12.0) * 100) && depth > 4 ? 0 :
      (rand() % productions.size());
12  }
13  else {
14    idx = productions.size() - 1;
15  }
16
17  if (depth > 16) {
18    for (int i = 0; i < (int)productions.size(); ++i) {
19      if (productions[i].empty()) { idx = i; break; }
20      if (productions[i].size() == 1 && rules.find(productions[i][0])
          == rules.end()) { idx = i; }
21    }
22  }
23
24  for (const string& sym : productions[idx]) {
25    generateNode(sym, output, depth + 1);
26  }
27 }

```

3.2.3 Метод parseNode

Данный метод используется для проверки корректности строки. Реализует алгоритм рекурсивного спуска с возвратом. Метод пытается применить каждое возможное правило для текущего определённого нетерминала и в случае если ни одно правило не подошло, происходит возврат к предыдущему нетерминалу, иначе – рекурсивно вызывается этот же метод для подходящего нетерминала.

Вход: введённая пользователем строка, текущая позиция каретки и проверяемое в данный момент правило грамматики.

Выход: true, если введённая строка является корректным предложением, иначе – false.

Код метода представлен на листинге 7.

Листинг 7. Код метода parseNode

```

1 bool parseNode(const vector<string>& tokens, size_t& pos, const string&
  symbol, TreeNode*& out_node) {
2   if (pos > max_parse_pos) max_parse_pos = pos;
3
4   if (rules.find(symbol) == rules.end()) {

```

```

5         if (pos < tokens.size() && tokens[pos] == symbol) {
6             out_node = new TreeNode(symbol);
7             pos++;
8             if (pos > max_parse_pos) max_parse_pos = pos;
9             return true;
10        }
11        return false;
12    }
13
14    size_t original_pos = pos;
15    for (const auto& prod : rules[symbol]) {
16        TreeNode* current = new TreeNode(symbol);
17        bool match = true;
18
19        if (prod.empty()) {
20            current->children.push_back(new TreeNode("ε"));
21        }
22        else {
23            for (const string& sym : prod) {
24                TreeNode* child = nullptr;
25                if (!parseNode(tokens, pos, sym, child)) {
26                    match = false;
27                    break;
28                }
29                if (child) current->children.push_back(child);
30            }
31        }
32
33        if (match) {
34            out_node = current;
35            return true;
36        }
37        delete current;
38        pos = original_pos;
39    }
40    return false;
41 }

```

3.2.4 Метод parseErrorTree

Метод служит для того, чтобы найти разобрать предложение и найти место ошибки в случае, если предложение оказалось некорректным. Также как и parseNode реализует рекурсивный спуск, но при невозможности использования правил вывода, записывает достигнутую глубину и делает возврат к предыдущему нетерминалу. После завершения обхода алгоритм выбирает максимальную достигнутую глубину, символ на которой помечает ошибочным.

Вход: введённая пользователем строка, текущая позиция каретки и проверяемое в данный момент правило грамматики.

Выход: код статуса (успешный / неуспешный / игнорирование ветки).

На листинге 8 представлен код данного метода.

```

1 int parseErrorTree(const vector<string>& tokens, size_t& pos, const
  string& symbol, TreeNode*& out_node) {
2   if (rules.find(symbol) == rules.end()) {
3       if (pos < tokens.size() && tokens[pos] == symbol) {
4           out_node = new TreeNode(symbol);
5           pos++;
6           return 1;
7       }
8       if (pos == max_parse_pos) {
9           string bad_word = (pos < tokens.size()) ? tokens[pos] : "КОНЕЦ"
10              ;
11           out_node = new TreeNode(COLOR_RED + "[ОЖИДАЛОСЬ:␣" + symbol + "
12              ,␣ВСТРЕЧЕНО:␣" + bad_word + "]" + COLOR_RESET);
13           return 2;
14       }
15       return 0;
16   }
17   size_t original_pos = pos;
18   for (const auto& prod : rules[symbol]) {
19       TreeNode* current = new TreeNode(symbol);
20       int status = 1;
21
22       if (prod.empty()) {
23           current->children.push_back(new TreeNode("ε"));
24       }
25       else {
26           for (const string& sym : prod) {
27               TreeNode* child = nullptr;
28               int res = parseErrorTree(tokens, pos, sym, child);
29               if (res == 1) {
30                   if (child) current->children.push_back(child);
31               }
32               else if (res == 2) {
33                   if (child) current->children.push_back(child);
34                   status = 2;
35                   break;
36               }
37               else {
38                   status = 0;
39                   break;
40               }
41           }
42
43           if (status == 1 || status == 2) {
44               out_node = current;
45               return status;
46           }
47           delete current;
48           pos = original_pos;
49       }
50       return 0;
51 }

```

3.2.5 Обработчики текста

На листинге 9 представлены исходные коды методов для обработки входных строк.

Листинг 9. Код методов для обработки текста

```
1 bool isCapitalized(const string& str) {
2     if (str.empty()) return false;
3     unsigned char c1 = str[0];
4     unsigned char c2 = str.size() > 1 ? str[1] : 0;
5     if (c1 == 0xC3 && c2 == 0x89) return true;
6     if (c1 >= 'A' && c1 <= 'Z') return true;
7     return false;
8 }
9
10 string lowercaseUtf8(const string& str) {
11     if (str.empty()) return str;
12     unsigned char c1 = str[0];
13     unsigned char c2 = str.size() > 1 ? str[1] : 0;
14     if (c1 == 0xC3 && c2 == 0x89) return "\xC3\xA9" + str.substr(2);
15     string res = str;
16     if (c1 >= 'A' && c1 <= 'Z') res[0] = c1 + 32;
17     return res;
18 }
19
20 string capitalizeUtf8(const string& str) {
21     if (str.empty()) return str;
22     unsigned char c1 = str[0];
23     unsigned char c2 = str.size() > 1 ? str[1] : 0;
24     if (c1 == 0xC3 && c2 == 0xA9) return "\xC3\x89" + str.substr(2);
25     string res = str;
26     if (c1 >= 'a' && c1 <= 'z') res[0] = c1 - 32;
27     return res;
28 }
```

Метод isCapitalized

Данный метод проверяет написана ли входная строка с заглавной буквы или начинается с «i» и заглавной буквы.

Вход: введённая строка

Выход: true, если строка написана с заглавной буквы или со знака «i» и идущей после него заглавной буквы, иначе – false.

Метод lowercaseUtf8

Метод lowercaseUtf8 приводит первый символ кроме «i» к нижнему регистру. Данный метод необходим для проведения правильного разбора введённого предложения, так как реализованная грамматика использует только слова с маленькой буквы.

Вход: введённая строка

Выход: введенная строка, начинающаяся со строчной буквы.

capitalizeUtf8

Данный метод выполняет функцию противоположную методу lowercaseUtf8 – он переводит первый символ предложения к верхнему регистру, так как во время обработки используется преобразованная строка, а при выводе ошибки необходимо вывести предложения в его изначальном виде.

Вход: введенная строка

Выход: введенная строка, начинающаяся с заглавной буквы.

3.2.6 Метод checkAndPrintSentence

Данный метод приводит введенную строку к виду, который программа сможет дальше обрабатывать:

- расставляет пробелы перед знаками препинания, чтобы программа могла выделить их как отдельные слова;
- проверяет наличие заглавной буквы в начале предложения;
- переводит первую букву в нижний регистр;
- запускает обработку строки через метод parseNode.

Вход: исходный текст предложения и настройка вывода дерева разбора

Выход: отформатированная строка и запущенная обработка полученной строки.

Код метода checkAndPrintSentence представлен на листинге 10.

Листинг 10. Код метода checkAndPrintSentence

```
1 void checkAndPrintSentence(string text, int tree_flag = 0) {
2     replaceAll(text, ".", "␣.");
3     replaceAll(text, "?", "␣?");
4     replaceAll(text, "!", "␣!");
5
6     vector<string> tokens;
7     stringstream ss(text);
8     string item;
9     while (ss >> item) tokens.push_back(item);
10
11     if (tokens.empty()) return;
12
13     int target_idx = (tokens[0] == "!") ? 1 : 0;
14     bool has_capital = false;
15     if (target_idx < tokens.size()) {
16         has_capital = isCapitalized(tokens[target_idx]);
17         if (has_capital) {
18             tokens[target_idx] = lowercaseUtf8(tokens[target_idx]);
19         }
20     }
```

```

21
22     if (!has_capital) {
23         cout << COLOR_RED << "[ОШИБКА]_" << COLOR_RESET << "Синтаксическ
            ая_ошибка._Предложение_должно_начинаться_с_заглавной_буквы.\n
            ";
24         return;
25     }
26
27     max_parse_pos = 0;
28     size_t pos = 0;
29     TreeNode* root = nullptr;
30     bool is_valid = parseNode(tokens, pos, "sentence", root);
31     is_valid = is_valid && (pos == tokens.size());
32
33     if (is_valid) {
34         cout << COLOR_GREEN << "[ОК]_" << COLOR_RESET << "Предложение_ко
            рректно.\n";
35         if (tree_flag != 0) {
36             cout << "Дерево_разбора:\n";
37             printTree(root);
38         }
39     }
40     else {
41         cout << COLOR_RED << "[ОШИБКА]_" << COLOR_RESET << "Синтаксическ
            ая_ошибка.\n";
42         cout << "Анализатор_остановился_на_позиции_" << max_parse_pos <<
            ":\n";
43
44         for (size_t i = 0; i < tokens.size(); ++i) {
45             string word_to_print = tokens[i];
46             // Исправление: восстанавливаем заглавную букву при выводе о
            шибки
47             if (i == target_idx) word_to_print = capitalizeUtf8(
                word_to_print);
48
49             if (i > 0 && tokens[i] != "." && tokens[i] != "?" && tokens[
                i - 1] != ";") cout << " ";
50
51             if (i == max_parse_pos) cout << COLOR_RED << word_to_print
                << COLOR_RESET;
52             else cout << word_to_print;
53         }
54         if (max_parse_pos == tokens.size()) cout << COLOR_RED << "_[КОНЕ
            Ц]" << COLOR_RESET;
55         cout << "\n";
56
57         if (tree_flag == 2) {
58             cout << "Частичное_дерево_разбора:\n";
59             TreeNode* err_root = nullptr;
60             size_t err_pos = 0;
61             parseErrorTree(tokens, err_pos, "sentence", err_root);
62             printTree(err_root);
63             delete err_root;
64         }
65     }
66     delete root;

```

3.2.7 Метод tokensToString

Данный метод преобразует полученный после генерации набор слов в предложение с правильным форматированием. В ходе выполнения функции производится расстановка пробелов между словами и перевод первого символа первого слова в верхний регистр. Код метода представлен на листинге 11.

Вход: вектор слов предложения.

Выход: отформатированное предложение, соответствующее поданному на вход вектору.

Листинг 11. Код метода tokensToString

```

1 string tokensToString(const vector<string>& tokens) {
2     string res = "";
3     bool capitalize_next = true;
4     for (size_t i = 0; i < tokens.size(); i++) {
5         if (i > 0 && tokens[i] != "." && tokens[i] != "?" && tokens[i - 1]
6             != ";") {
7             res += " ";
8             string current_token = tokens[i];
9             if (capitalize_next && current_token != ";") {
10                current_token = capitalizeUtf8(current_token);
11                capitalize_next = false;
12            }
13            res += current_token;
14        }
15        return res;
16    }

```

3.3 Функция обработки пользовательского ввода

В функции userIO реализовано взаимодействие программы с пользователем с помощью консоли. После запуска программы пользователю выводится главное меню, в котором ему предлагается ввести предложение, сгенерировать предложения или изменить настройку вывода дерева.

Если пользователь выбрал ввести предложение, программа ожидает ввода от него и проводит проверку корректности введённой строки, после чего выводит результаты проверки и дерево разбора в зависимости от заданной пользователем настройки.

Если пользователь выбирает сгенерировать предложения то у него запрашивается количество необходимых предложений и вероятность генерации неверного предложения, задаваемая в процентах. После этого программа генерирует заданное количество предложений и выводит пользователю информацию об их корректности также с выводом дерева разбора.

Для настройки вывода дерева разбора предложения пользователь выбирает соответствующий пункт в главном меню. При каждом выборе данного действия циклически меняется настройка вывода дерева – "Нет", "Только для корректных" и "Да".

Вход: экземпляр КС-грамматики (класс LanguageModel).

Выход: вывод меню пользователю и выполнение действий выбранных пользователем.

Код данной функции представлен на листинге 12.

Листинг 12. Код функции main

```

1 int userIO(Language model) {
2     int choice;
3     bool running = true;
4
5     while (running) {
6         cout << "\n===== \n";
7         cout << "        Главное меню        \n";
8         cout << "1. Ввести предложение \n";
9         cout << "2. Сгенерировать предложения \n";
10        cout << "3. Выводить дерево: \n" << ((treeNeeded == 0) ? "Нет" : ((
            treeNeeded == 1) ? "Только для корректных" : "Для всех")) <<
            endl;
11        cout << "4. Выход \n";
12        cout << "===== \n";
13
14        if (!readInt(choice, "Выберите действие: \n", 1, 4)) {
15            cout << "Введите число от 1 до 4. \n";
16            continue;
17        }
18
19        if (choice == 1) {
20            cout << "\nВведите предложение на испанском: \n> \n";
21            string input;
22            getline(cin, input);
23            model.checkAndPrintSentence(input, treeNeeded);
24        }
25        else if (choice == 2) {
26            int n, m;
27            readInt(n, "Количество строк: \n", 1, 100);
28            readInt(m, "Вероятность ошибки (0-100): \n", 0, 100);
29
30            for (int i = 0; i < n; ++i) {
31                vector<string> tokens = model.generateTokenList();
32                if ((rand() % 100) < m && !tokens.empty()) {
33                    int attempts = 0;
34                    while (attempts++ < 10) {
35                        int idx = rand() % tokens.size();
36                        if (tokens[idx] != "." && tokens[idx] != "?" && tokens
                            [idx] != "?") {
37                            tokens[idx] = model.getRandomWord();
38                            break;
39                        }
40                    }
41                }
42                string sentence = model.tokensToString(tokens);

```

```

43         cout << "\nCтpoka_" << i + 1 << ":_" << sentence << "\n";
44         model.checkAndPrintSentence(sentence, treeNeeded);
45     }
46 }
47 else if (choice == 3) {
48     treeNeeded = (treeNeeded + 1) % 3;
49 }
50 else if (choice == 4) {
51     running = false;
52 }
53 }
54 }

```

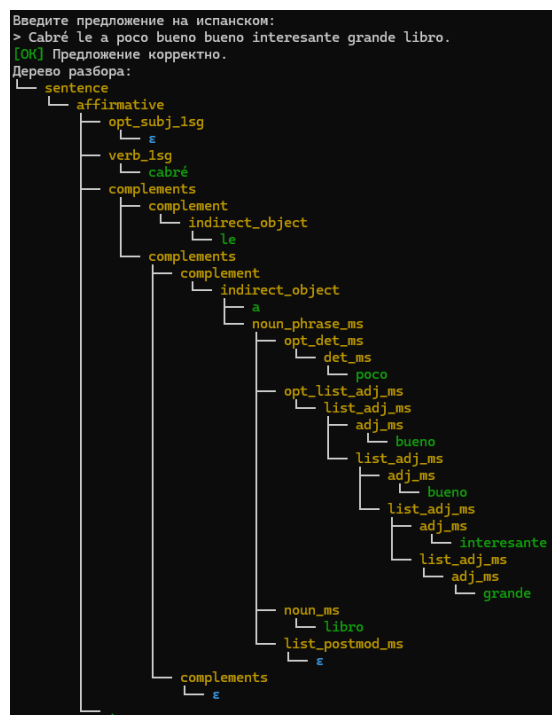
4 Результаты работы программы

На рисунке 2 продемонстрирована обработка не корректного ввода при задании параметров генерации предложений.

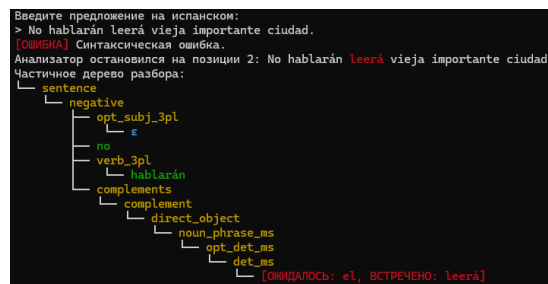
```
=====
                          Главное меню
1. Ввести предложение
2. Сгенерировать предложения
3. Выводить дерево: Нет
4. Выход
=====
Выберите действие: 2
Количество строк: 1001
Введите число от 1 до 100.
Количество строк: 17
Вероятность ошибки (0-100): -15
Введите число от 0 до 100.
Вероятность ошибки (0-100): абв
Введите число от 0 до 100.
Вероятность ошибки (0-100): 0
=====
```

Рис. 2. Обработка некорректного пользовательского ввода

Примеры ввода пользователем соответствующего и несоответствующего предложений представлены на рисунках 3а и 3б соответственно. При вводе правильной строки выводится полное дерево разбора предложения, иначе – только до непринятого слова.



(a) Соответствующее предложение



(b) Несоответствующее предложение

Рис. 3. Вывод программы при вводе предложения пользователем

Также на рисунке 4 представлен ввод пользователем корректного утвердительного, отрицательного и вопросительного предложений.

```
=====
                Главное меню
1. Ввести предложение
2. Сгенерировать предложения
3. Выводить дерево: Нет
4. Выход
=====
Выберите действие: 1

Введите предложение на испанском:
> Habréis a muchos importantes perros que pondrán que comerás a todas vidas lo.
[OK] Предложение корректно.

=====
                Главное меню
1. Ввести предложение
2. Сгенерировать предложения
3. Выводить дерево: Нет
4. Выход
=====
Выберите действие: 1

Введите предложение на испанском:
> Tú no dirás les mañana a gatos que saldrás les lo a poca puerta.
[OK] Предложение корректно.

=====
                Главное меню
1. Ввести предложение
2. Сгенерировать предложения
3. Выводить дерево: Нет
4. Выход
=====
Выберите действие: 1

Введите предложение на испанском:
> ¿Tendrá usted a esta grande puerta que trabajaremos les a escuelas de tus viejas profesoras las?
[OK] Предложение корректно.
```

Рис. 4. Ввод утвердительного, отрицательного и вопросительного предложений

Далее на рисунках 5 и 6 представлена генерация предложений без вывода деревьев разбора 10 строк с 50% некорректных строк и с 0% соответственно.

```
Выберите действие: 2
Количество строк: 10
Вероятность ошибки (0-100): 50

Строка 1: No podrán los.
[OK] Предложение корректно.

Строка 2: ¿Qué trabajarás tú aquí a libro vieja vieja amiga nunca seguramente a tiempos?
[OK] Предложение корректно.

Строка 3: Estudiaréis no cabréis.
[ОШИБКА] Синтаксическая ошибка.
Анализатор остановился на позиции 1: Estudiaréis no cabréis.

Строка 4: ¿Todas siempre lo a tus mesas a tu vida bajo toda roja buena profesora hasta una ciudad a poca mano para tu hermosa pequeña ciudad sobre la hermosa buena hermosa hermosa noche?
[ОШИБКА] Синтаксическая ошибка.
Анализатор остановился на позиции 1: ¿Todas siempre lo a tus mesas a tu vida bajo toda roja buena profesora hasta una ciudad a poca mano para tu hermosa pequeña ciudad sobre la hermosa buena hermosa hermosa noche?

Строка 5: ¿Cuánta habrá la la a puerta que valdrá?
[OK] Предложение корректно.

Строка 6: ¿Leeré aquí a nuestras inteligentes vidas a gatos desde nuevos interesantes hermosos importantes coches los esta ciudad?
[OK] Предложение корректно.

Строка 7: Yo nuevas seguramente siempre rojas nuevas flores mis nuevos problemas.
[ОШИБКА] Синтаксическая ошибка.
Анализатор остановился на позиции 1: Yo nuevas seguramente siempre rojas nuevas flores mis nuevos problemas.

Строка 8: Diréis a vosotros grande profesor quizás le.
[ОШИБКА] Синтаксическая ошибка.
Анализатор остановился на позиции 2: Diréis a vosotros grande profesor quizás le.

Строка 9: ¿Trabajaréis?
[OK] Предложение корректно.

Строка 10: Yo escuelas siempre.
[ОШИБКА] Синтаксическая ошибка.
Анализатор остановился на позиции 1: Yo escuelas siempre.
```

Рис. 5. Генерация 10 предложений с 50% некорректных

```
Выберите действие: 2
Количество строк: 10
Вероятность ошибки (0-100): 0

Строка 1: ¿Cuántos escribirá a libro poca nueva amiga que valdré en puerta que viviremos?
[OK] Предложение корректно.

Строка 2: No correrás a pequeño coche con un interesante árbol que valdremos la la los los a muchos tiempos estas import
antes hermosas hermosas pequeñas ciudades.
[OK] Предложение корректно.

Строка 3: No trabajaréis mal a árboles nunca mañana.
[OK] Предложение корректно.

Строка 4: Yo no bailaré a mi casa que hará que recibirán seguramente mi pequeño coche que valdrá le aquí bien a este tra
bajo.
[OK] Предложение корректно.

Строка 5: ¿Quién querremos nosotras los los mañana pronto tus grandes inteligentes rojos profesores libro mañana?
[OK] Предложение корректно.

Строка 6: ¿Cómo pondré les mañana la a esta flor?
[OK] Предложение корректно.

Строка 7: ¿Tendré yo le allí les interesante interesante casa les los?
[OK] Предложение корректно.

Строка 8: Amiga sobre vieja ciudad entre poca grande mano que correremos nuestros trabajos dirá a vuestros importantes t
rabajos que trabajarás mañana las desde tus interesantes pequeños árboles.
[OK] Предложение корректно.

Строка 9: ¿Diréis tiempo mal lo perros entre trabajos desde vuestros árboles a tus árboles a el profesor sin importante
árbol por el problema rápidamente le?
[OK] Предложение корректно.

Строка 10: Yo recibiré a coche por su trabajo bajo rápido gato que escribiremos a hermosa inteligente importante amiga t
ambién también a esta rápida grande buena interesante puerta que querré.
[OK] Предложение корректно.
```

Рис. 6. Генерация 10 предложений с 0% некорректных

Заключение

В ходе выполнения данной лабораторной работы была разработана контекстно-свободная грамматика для подмножества будущего простого времени испанского языка.

Далее по данной грамматике были разработаны анализатор введённых пользователем предложений и генератор предложений, который поддерживает задание необходимого соотношения некорректных предложений к корректным.

Разработанная программа правильно обрабатывает и генерирует предложения из выбранного языка, а также может выводить дерево разбора предложения при работе с ним.

Достоинства:

- Разработанный класс `LanguageModel` позволяет создавать анализаторы и генераторы любых КС грамматик.
- КС грамматики можно задавать с помощью простого перечисления правил вывода.
- Реализована генерации деревьев разбора предложений как при генерации, так и при валидации.
- При валидации и генерации предложений выводятся их деревья разбора. В случае, если предложение неправильное, также будет выводиться одно из слов, которые программа ожидала встретить на позиции ошибочного слова.

Недостатки:

- Для генерации и валидации предложений используется метод рекурсивного спуска с откатами, который выполняется с экспоненциальной сложностью по времени.
- При генерации предложений не учитывается их семантика из-за чего могут получаться семантически неправильные фразы.

Масштабирование:

- Переписать класс грамматики так, чтобы внутри он хранил и использовал конечный автомат со стеком, представленный экземпляром класса `FSM`, разработанного в первой лабораторной работе.

Список использованной литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 29.03.2026).
- [2] Вирт, Н. Построение компиляторов / Н. Вирт. — М.: ДМК Пресс, 2010. — 190 с.
- [3] Ахо, А. В. Компиляторы: принципы, технологии и инструментарий [2-е изд.] / А. В. Ахо, М. С. Лам, Р. Сети, Дж. Д. Ульман ; Пер. с англ. — М. : ООО «И.Д. Вильямс», 2018. — 1184 с.